DATABASE MANAGEMENT SYSTEM LAB

LAB SHEET

Submitted in partial fulfilment of the requirement of the degree of

BACHELORS OF TECHNOLOGY

in

CSE CORE (SECTION-A)

to

K.R Mangalam University

SUBMITTED BY:

MONISHKA BAJAJ

2401010161

SUBMITTED TO:

MS. MANSI KAJAL

Department of Computer Science and Engineering

School of Engineering and Technology

K.R Mangalam University, Gurugram- 122001, India

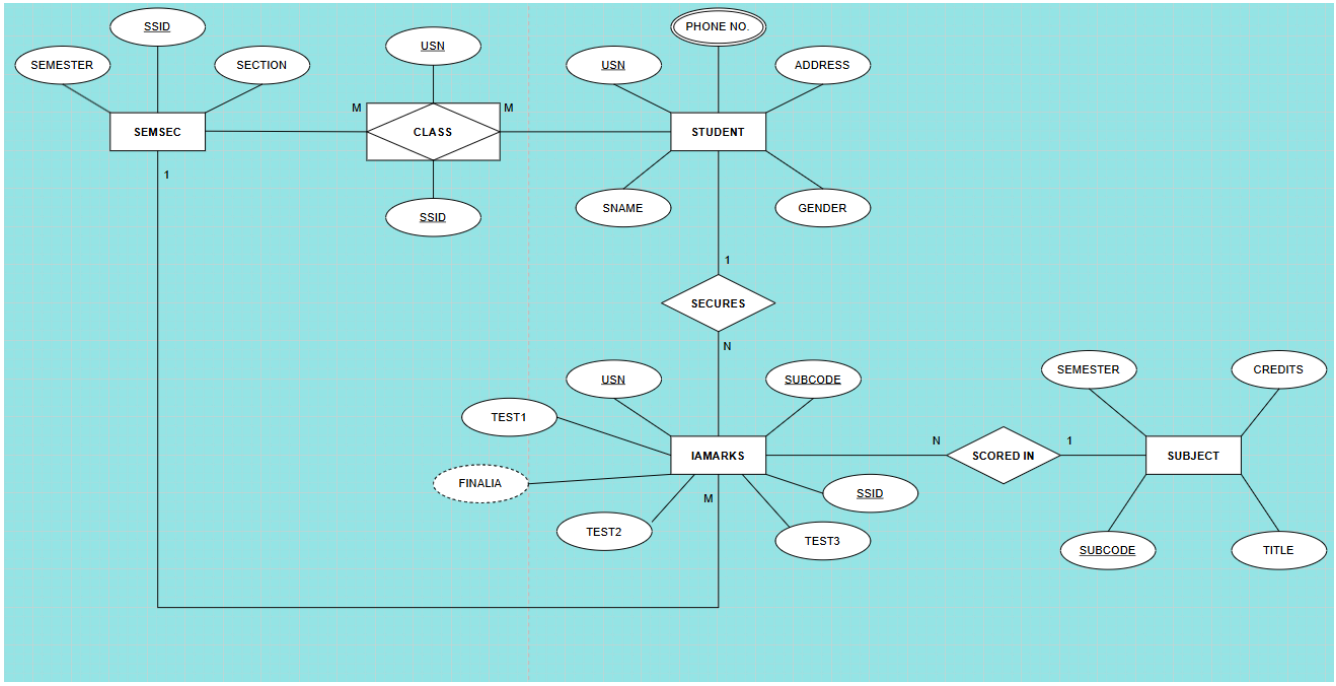
SESSION – 2025-2026

EXPERIMENT-1

PART A – COLLEGE DATABASE

STUDENT (USN, SName, Address, Phone, Gender)
SEMSEC (SSID, Sem, Sec)
CLASS (USN, SSID)
SUBJECT (Subcode, Title, Sem, Credits)
IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

ER DIAGRAM – COLLEGE DATABASE



This ER (Entity-Relationship) diagram represents a Student Information System. It describes the relationship between **STUDENT, SUBJECT, SEMSEC, IAMARKS, and CLASS** entities.

Entities and Attributes:

1. **STUDENT Entity**
   Stores student details.
   Attributes:
   - USN (Primary Key)
   - SNAME
   - PHONE NO.
   - ADDRESS
   - GENDER

2. **SUBJECT Entity**
   Stores subject information.
   Attributes:
   - SUBCODE (Primary Key)
   - TITLE
   - SEMESTER
   - CREDITS
3. **SEMSEC Entity**
   Stores semester and section details.
   Attributes:
   - SSID (Primary Key)
   - SEMESTER
   - SECTION
4. **IAMARKS Entity**
   Stores internal assessment marks of students.
   Attributes:
   - USN (Foreign Key)
   - SUBCODE (Foreign Key)
   - SSID
   - TEST1
   - TEST2
   - TEST3
   - FINALIA

Relationships:

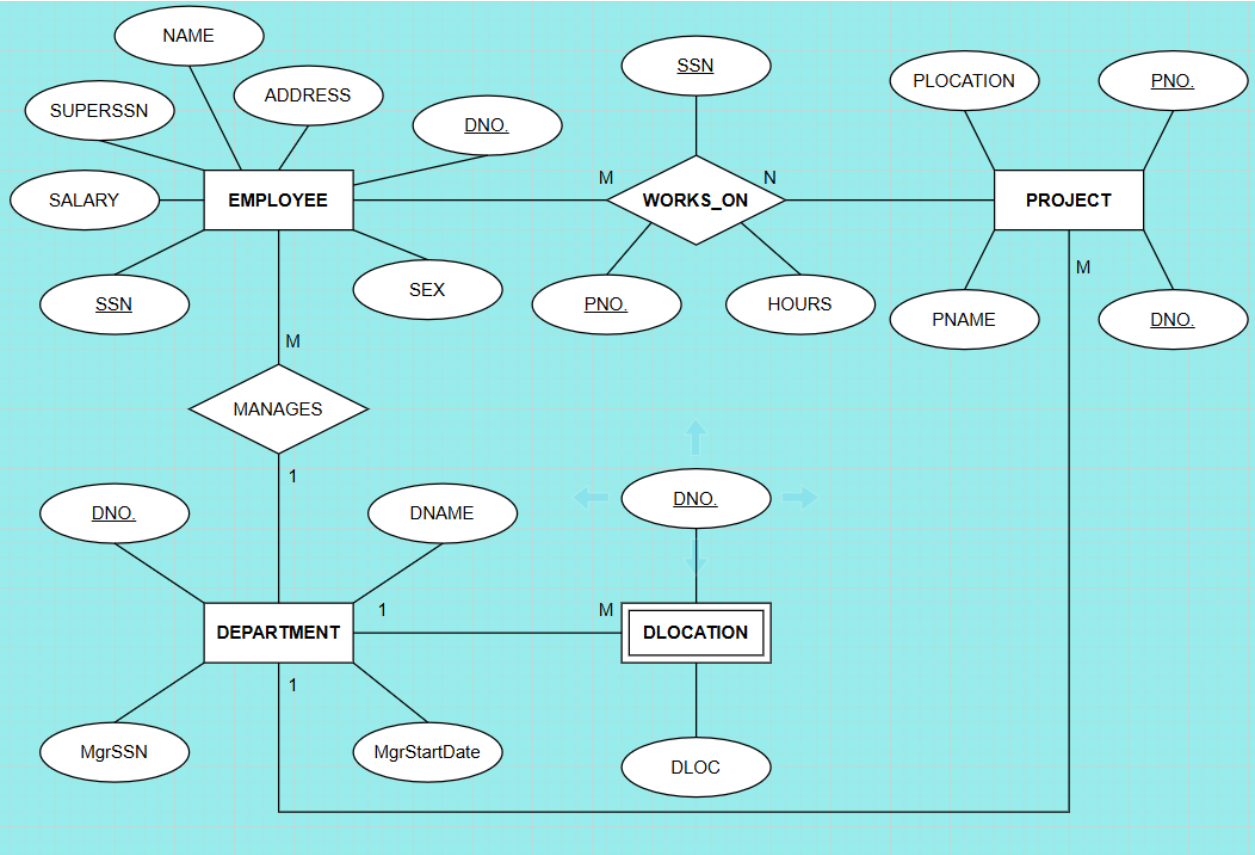
- **CLASS (M:N Relationship)**
  Connects STUDENT and SEMSEC. A student can belong to a class, and each class can have multiple students.
- **SECURES (1:N Relationship)**
  A student secures multiple IA marks records.
- **SCORED IN (N:1 Relationship)**
  IAMARKS are scored in a particular SUBJECT.

This ER diagram helps in understanding how students, subjects, classes, and internal assessment marks are related in a student database system.

PART B – COMPANY DATABASE

EMPLOYEE (SSN, Name, Address, Sex, Salary, Superssn, Dno)
DEPARTMENT (Dno, Dname, Mgrssn, Mgrstartdate)
DLOCATION (Dno, Dloc)
PROJECT (PNo, PName, PLocation, DNo)
WORKS_ON (SSN, PNo, Hours)

ER DIAGRAM – COMPANY DATABASE



This ER (Entity-Relationship) diagram represents a Company Database System. It describes the relationship between **EMPLOYEE, DEPARTMENT, PROJECT, and DLOCATION** entities.

Entities and Attributes:

1. **EMPLOYEE Entity**
   Stores information about employees.
   Attributes:
   - SSN (Primary Key)
   - Name
   - Address
   - Salary

    - Sex
    - SuperSSN
    - DNO (Department Number – Foreign Key)

2. **DEPARTMENT Entity**
   Stores department details.
   Attributes:
    - DNO (Primary Key)
    - DNAME
    - MgrSSN
    - MgrStartDate

3. **PROJECT Entity**
   Stores project information.
   Attributes:
    - PNO (Primary Key)
    - PNAME
    - PLOCATION
    - DNO (Foreign Key)

4. **DLOCATION Entity**
   Stores department locations.
   Attribute:
    - DLOC

Relationships:

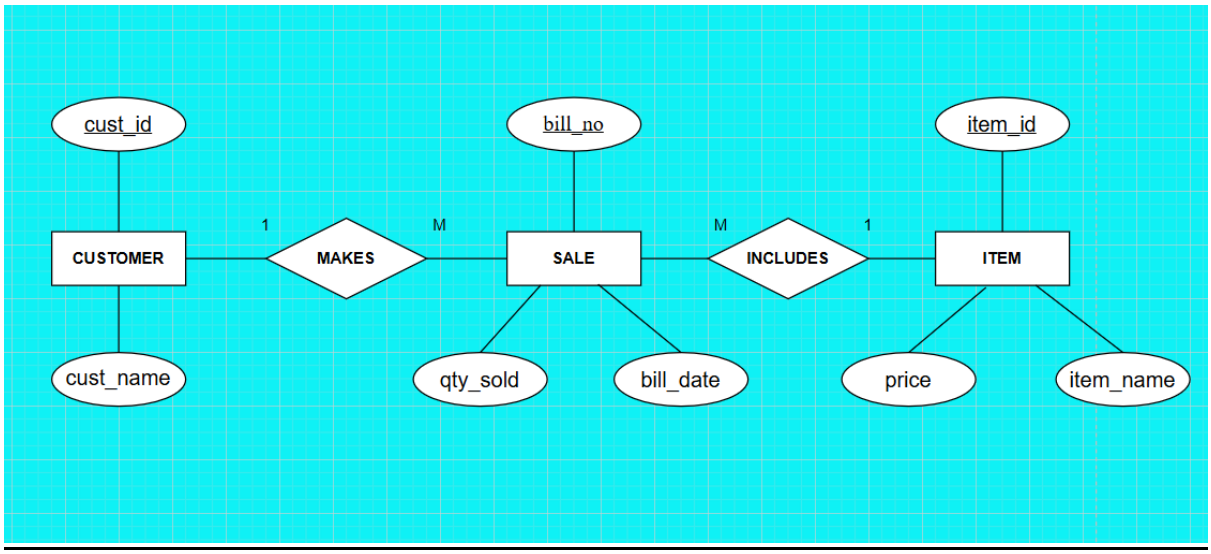
- **WORKS_ON (M:N Relationship)**
  An employee can work on multiple projects, and a project can have multiple employees.
  It has an attribute:
    - Hours (Number of hours worked)
- **MANAGES (1:M Relationship)**
  A department is managed by one employee, and an employee can manage a department.
- **DEPARTMENT – DLOCATION (1:M Relationship)**
  One department can have multiple locations.

This ER diagram helps in understanding how employees, departments, and projects are related within a company database system.

PART C – CUSTOMER SALE SCHEMA

- **Database Schema for a customer-sale scenario**
- **Customer(Cust id : integer, cust_name: string)**
- **Item(item_id: integer, item_name:string, price: integer)**
  Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

ER DIAGRAM – CUSTOMER SALE SCHEMA



This ER (Entity-Relationship) diagram represents a Sales Management System. It shows the relationship between three main entities: **CUSTOMER, SALE, and ITEM**.

1. **CUSTOMER Entity**
   This entity stores customer information.
   Attributes:
   - cust_id (Primary Key)
   - cust_name
2. **SALE Entity**
   This entity stores details of each sale transaction.
   Attributes:
   - bill_no (Primary Key)
   - bill_date
   - qty_sold
3. **ITEM Entity**
   This entity stores product details.
   Attributes:

- item_id (Primary Key)
- item_name
- price

Relationships with Cardinality:

1. **MAKES (1:M Relationship)**
   One CUSTOMER can make many SALES, but each SALE is made by only one CUSTOMER.
2. **INCLUDES (M:1 Relationship)**
   Many SALES can include one ITEM, but each ITEM entry in this relationship refers to a single item record.

This ER diagram helps in understanding how customers, sales, and items are connected in a sales system database.

DDL Questions (Data Definition Language):

1.Write SQL statements to create all the above relations with appropriate PRIMARY KEY and FOREIGN KEY constraints.

```sql
CREATE DATABASE MONISHKA_ASSIGNMENT;
USE MONISHKA_ASSIGNMENT;
```

PART-A COLLEGE DATABASE:

1.CREATE TABLES:

```sql
CREATE TABLE STUDENT (
    USN VARCHAR(20) PRIMARY KEY,
    SName VARCHAR(50),
    Address VARCHAR(100),
    Phone VARCHAR(15),
    Gender CHAR(1)
);
```

```sql
CREATE TABLE SEMSEC (
    SSID VARCHAR(10) PRIMARY KEY,
    Sem INT,
    Sec CHAR(1)
);
```

```sql
CREATE TABLE CLASS (
    USN VARCHAR(20),
    SSID VARCHAR(10),
    PRIMARY KEY (USN, SSID),
    FOREIGN KEY (USN) REFERENCES STUDENT(USN),
    FOREIGN KEY (SSID) REFERENCES SEMSEC(SSID)
);
```

```sql
CREATE TABLE SUBJECT (
    Subcode VARCHAR(10) PRIMARY KEY,
    Title VARCHAR(50),
    Sem INT,
    Credits INT
);
```

```sql
CREATE TABLE IAMARKS (
    USN VARCHAR(20),
    Subcode VARCHAR(10),
    SSID VARCHAR(10),
    Test1 INT,
    Test2 INT,
    Test3 INT,
    FinalIA INT,
    PRIMARY KEY (USN, Subcode, SSID),
    FOREIGN KEY (USN) REFERENCES STUDENT(USN),
    FOREIGN KEY (Subcode) REFERENCES SUBJECT(Subcode),
    FOREIGN KEY (SSID) REFERENCES SEMSEC(SSID)
);
```

PART- B COMPANY DATABASE:

```sql
CREATE TABLE DEPARTMENT (
    Dno INT PRIMARY KEY,
    Dname VARCHAR(50),
    Mgrssn VARCHAR(20),
    Mgrstartdate DATE
);
```

```sql
CREATE TABLE EMPLOYEE (
    SSN VARCHAR(20) PRIMARY KEY,
    Name VARCHAR(50),
    Address VARCHAR(100),
    Sex CHAR(1),
    Salary DECIMAL(10,2) NOT NULL,
    Superssn VARCHAR(20),
    Dno INT,
    FOREIGN KEY (Superssn) REFERENCES EMPLOYEE(SSN),
    FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dno)
);
```

```sql
CREATE TABLE DLOCATION (
    Dno INT,
    Dloc VARCHAR(50),
    PRIMARY KEY (Dno, Dloc),
    FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dno)
);
```

```sql
CREATE TABLE PROJECT (
    PNo INT PRIMARY KEY,
    PName VARCHAR(50),
    PLocation VARCHAR(50),
    DNo INT,
    FOREIGN KEY (DNo) REFERENCES DEPARTMENT(Dno)
);
```

```sql
CREATE TABLE WORKS_ON (
    SSN VARCHAR(20),
    PNo INT,
    Hours DECIMAL(5,2),
    PRIMARY KEY (SSN, PNo),
    FOREIGN KEY (SSN) REFERENCES EMPLOYEE(SSN),
    FOREIGN KEY (PNo) REFERENCES PROJECT(PNo)
);
```

PART-C CUSTOMER SALE SCHEMA DATABASE:

```sql
CREATE TABLE CUSTOMER (
    Cust_id INT PRIMARY KEY,
    cust_name VARCHAR(50)
);
```

```sql
CREATE TABLE ITEM (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(50),
    price INT NOT NULL
);
```

```sql
CREATE TABLE SALE (
    bill_no INT,
    bill_date DATE,
    cust_id INT,
    item_id INT,
    qty_sold INT,
    PRIMARY KEY (bill_no, item_id),
    FOREIGN KEY (cust_id) REFERENCES CUSTOMER(Cust_id),
    FOREIGN KEY (item_id) REFERENCES ITEM(item_id)
);
```

---

2. Add a new column Email to the STUDENT relation.

```sql
ALTER TABLE STUDENT
ADD Email VARCHAR(50);
```

Result Grid | Filter Rows: | Export:

Field	Type	Null	Key	Default	Extra
USN	varchar(20)	NO	PRI	NULL	
SName	varchar(50)	YES		NULL	
Address	varchar(100)	YES		NULL	
Phone	varchar(15)	YES		NULL	
Gender	char(1)	YES		NULL	
Email	varchar(50)	YES		NULL	

3. Apply NOT NULL constraints to Salary and Price attributes.

```
CREATE TABLE EMPLOYEE (
    SSN VARCHAR(20) PRIMARY KEY,
    Name VARCHAR(50),
    Address VARCHAR(100),
    Sex CHAR(1),
    Salary DECIMAL(10,2) NOT NULL,
    Superssn VARCHAR(20),
    Dno INT,
    FOREIGN KEY (Superssn) REFERENCES EMPLOYEE(SSN),
    FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dno)
);
```

IN EMPLOYEE TABLE SALARY IS NOT NULL

```
CREATE TABLE ITEM (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(50),
    price INT NOT NULL
);
```

IN ITEM TABLE PRICE IS NOT NULL

4. Rename attribute SName to StudentName in STUDENT relation.

```
ALTER TABLE STUDENT
RENAME COLUMN SName TO StudentName;
```

Result Grid | Filter Rows: | Export: | Wrap

Field	Type	Null	Key	Default	Extra
USN	varchar(20)	NO	PRI	NULL	
StudentName	varchar(50)	YES		NULL	
Address	varchar(100)	YES		NULL	
Phone	varchar(15)	YES		NULL	
Gender	char(1)	YES		NULL	
Email	varchar(50)	YES		NULL	

---

5. Drop the DLOCATION relation.

```
DROP TABLE DLOCATION;
```

DML Questions (Data Manipulation Language):

Question- Database Schema for a customer-sale scenario

- **Customer(Cust id : integer, cust_name: string)**
- **Item(item_id: integer, item_name: string, price: integer)**
- **Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)**

For the above schema, perform the following—

```
CREATE DATABASE customer_sale;
USE customer_sale;
```

1. Create the tables with the appropriate integrity constraints.

```
CREATE TABLE Customer (
    cust_id INT PRIMARY KEY,
    cust_name VARCHAR(50) NOT NULL
);

CREATE TABLE Item (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(50) NOT NULL,
    price INT NOT NULL CHECK (price > 0)
);

CREATE TABLE Sale (
    bill_no INT PRIMARY KEY,
    bill_date DATE NOT NULL,
    cust_id INT,
    item_id INT,
    qty_sold INT NOT NULL CHECK (qty_sold > 0),
    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
    FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

2. Insert around 10 records in each of the tables.

```sql
INSERT INTO Customer VALUES
(1,'Aman'),
(2,'Riya'),
(3,'Karan'),
(4,'Sneha'),
(5,'Rahul'),
(6,'Priya'),
(7,'Vikas'),
(8,'Neha'),
(9,'Arjun'),
(10,'Pooja');
INSERT INTO Item VALUES
(101,'Pen',20),
(102,'Notebook',50),
(103,'Bag',800),
(104,'Shoes',1200),
(105,'Watch',1500),
(106,'Bottle',150),
(107,'Headphones',700),
(108,'Keyboard',900),
(109,'Mouse',400),
(110,'Charger',300);
```

```sql
INSERT INTO Sale VALUES
(1001,CURDATE(),1,101,5),
(1002,CURDATE(),2,103,1),
(1003,CURDATE(),3,104,2),
(1004,CURDATE(),4,102,3),
(1005,CURDATE(),5,105,1),
(1006,CURDATE()-1,6,109,2),
(1007,CURDATE()-2,7,108,1),
(1008,CURDATE()-3,8,110,4),
(1009,CURDATE(),9,107,2),
(1010,CURDATE(),10,106,6);
```

3. List all the bills for the current date with the customer names and item numbers.

```sql
SELECT s.bill_no,
       s.bill_date,
       c.cust_name,
       s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();
```

Result Grid | Filter Rows:

bill_no	bill_date	cust_name	item_id
1001	2026-02-22	Aman	101
1002	2026-02-22	Riya	103
1003	2026-02-22	Karan	104
1004	2026-02-22	Sneha	102
1005	2026-02-22	Rahul	105
1009	2026-02-22	Arjun	107
1010	2026-02-22	Pooja	106

4. List the total Bill details with the quantity sold, price of the item and the final amount.

```
SELECT s.bill_no,
       c.cust_name,
       i.item_name,
       s.qty_sold,
       i.price,
       (s.qty_sold * i.price) AS final_amount
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
JOIN Item i ON s.item_id = i.item_id;
```

Result Grid | Filter Rows: | Export: | Wrap Ce

bill_no	cust_name	item_name	qty_sold	price	final_amount
1001	Aman	Pen	5	20	100
1002	Riya	Bag	1	800	800
1003	Karan	Shoes	2	1200	2400
1004	Sneha	Notebook	3	50	150
1005	Rahul	Watch	1	1500	1500
1006	Priya	Mouse	2	400	800
1007	Vikas	Keyboard	1	900	900
1008	Neha	Charger	4	300	1200
1009	Arjun	Headphones	2	700	1400
1010	Pooja	Bottle	6	150	900

5.List the total Bill details with the quantity sold, price of the item and the final amount.

```
SELECT DISTINCT c.cust_id,
       c.cust_name
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

Result Grid | Filter Rows:

cust_id	cust_name
2	Riya
3	Karan
5	Rahul
9	Arjun
7	Vikas
6	Priya
8	Neha

6. Give a count of how many products have been bought by each customer

```
SELECT c.cust_id,
       c.cust_name,
       SUM(s.qty_sold) AS total_products
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_id, c.cust_name;
```

Result Grid | Filter Rows:

cust_id	cust_name	total_products
1	Aman	5
2	Riya	1
3	Karan	2
4	Sneha	3
5	Rahul	1
6	Priya	2
7	Vikas	1
8	Neha	4
9	Arjun	2
10	Pooja	6

7. Give a count of how many products have been bought by each customer

```
SELECT i.item_name,
       s.qty_sold
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;
```

Result Grid | Filter Rows:

item_name	qty_sold
Watch	1

8. List the item details which are sold as of today.

```
SELECT DISTINCT i.item_id,
       i.item_name,
       i.price
FROM Item i
JOIN Sale s ON i.item_id = s.item_id
WHERE s.bill_date = CURDATE();
```

Result Grid | Filter Rows:

item_id	item_name	price
101	Pen	20
103	Bag	800
104	Shoes	1200
102	Notebook	50
105	Watch	1500
107	Headphones	700
106	Bottle	150

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

```
CREATE VIEW Bill_Details AS
SELECT s.bill_no,
       s.bill_date,
       s.cust_id,
       s.item_id,
       i.price,
       s.qty_sold,
       (i.price * s.qty_sold) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
SELECT * FROM Bill_Details;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
1001	2026-02-22	1	101	20	5	100
1002	2026-02-22	2	103	800	1	800
1003	2026-02-22	3	104	1200	2	2400
1004	2026-02-22	4	102	50	3	150
1005	2026-02-22	5	105	1500	1	1500
1006	2026-02-21	6	109	400	2	800
1007	2026-02-20	7	108	900	1	900
1008	2026-02-19	8	110	300	4	1200
1009	2026-02-22	9	107	700	2	1400
1010	2026-02-22	10	106	150	6	900

10. Create a view which lists the daily sales date wise for the last one week

```
CREATE VIEW Weekly_Sales AS
SELECT s.bill_date,
       SUM(i.price * s.qty_sold) AS total_daily_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY s.bill_date;
SELECT * FROM Weekly_Sales;
```

Result Grid Filter Rows:

bill_date	total_daily_sales
2026-02-22	7250
2026-02-21	800
2026-02-20	900
2026-02-19	1200